

# AI Demo OffGas

Documentazione tecnica e operativa della proof of concept ML

Progetto OffGas

Maggio 2026

## Indice

|                                                                          |          |
|--------------------------------------------------------------------------|----------|
| <b>1. Scopo del documento</b>                                            | <b>2</b> |
| <b>2. Ruolo della demo AI nel progetto OffGas</b>                        | <b>2</b> |
| <b>3. Posizionamento rispetto all'architettura attuale</b>               | <b>2</b> |
| <b>4. File presenti nella cartella</b>                                   | <b>3</b> |
| <b>5. Il dataset <code>gas_data.csv</code></b>                           | <b>3</b> |
| <b>6. Obiettivo del modello</b>                                          | <b>3</b> |
| <b>7. Scelta del modello: <code>RandomForestRegressor</code></b>         | <b>4</b> |
| <b>8. Trasformazione della serie temporale in dataset supervisionato</b> | <b>4</b> |
| <b>9. Orizzonte temporale della predizione</b>                           | <b>4</b> |
| <b>10. Processo di training in <code>train_randomforest.py</code></b>    | <b>5</b> |
| <b>11. Valutazione del modello</b>                                       | <b>5</b> |
| <b>12. Salvataggio del modello: <code>gas_rf_model.pkl</code></b>        | <b>5</b> |
| <b>13. Processo di inferenza in <code>predict_demo.py</code></b>         | <b>6</b> |
| <b>14. Soglia dimostrativa e <code>predicted_crossing</code></b>         | <b>6</b> |
| <b>15. Come eseguire la demo</b>                                         | <b>6</b> |
| 15.1 Installazione librerie . . . . .                                    | 6        |
| 15.2 Training del modello . . . . .                                      | 7        |
| 15.3 Esecuzione della predizione . . . . .                               | 7        |
| <b>16. Uso di un nuovo <code>gas_data.csv</code></b>                     | <b>7</b> |
| 16.1 Nuovo CSV usato solo per inferenza . . . . .                        | 7        |
| 16.2 Nuovo CSV usato anche per riaddestramento . . . . .                 | 7        |
| <b>17. Cosa è reale e cosa è dimostrativo nella demo</b>                 | <b>7</b> |
| 17.1 Elementi reali . . . . .                                            | 7        |
| 17.2 Elementi dimostrativi . . . . .                                     | 8        |
| <b>18. Limiti intenzionali della demo</b>                                | <b>8</b> |

|                                 |   |
|---------------------------------|---|
| 19. Possibile evoluzione futura | 8 |
| 20. Sintesi finale              | 8 |

## 1. Scopo del documento

Questo documento descrive la cartella **offgas\_ai\_demo** del progetto OffGas nella sua forma attuale.

L'obiettivo è spiegare:

- quale ruolo svolge la demo AI rispetto al progetto principale;
- quali file contiene la cartella e a cosa servono;
- come viene costruita la pipeline di training e predizione;
- quali feature temporali vengono usate;
- come viene interpretata la predizione a circa **30 secondi**;
- quali sono i limiti intenzionali della demo;
- in che modo questa proof of concept potrebbe evolvere in una futura integrazione lato server.

La cartella viene trattata come una **proof of concept separata**, pensata per mostrare una possibile evoluzione futura del progetto OffGas tramite Machine Learning, senza modificare il runtime reale del sistema.

## 2. Ruolo della demo AI nel progetto OffGas

La cartella **offgas\_ai\_demo** non sostituisce il progetto principale e non cambia il funzionamento corrente di:

- Arduino;
- Bridge Python;
- broker MQTT;
- Node-RED;
- dashboard web.

Il suo ruolo è esclusivamente dimostrativo. Serve a mostrare che i dati storici raccolti da OffGas possono essere riutilizzati per costruire una prima pipeline di **predizione del valore futuro del gas di G1**.

L'idea architetturale è quindi la seguente:

Progetto principale:

Arduino -> Bridge -> MQTT -> Node-RED -> Dashboard

Demo AI separata:

gas\_data.csv -> train\_randomforest.py -> gas\_rf\_model.pkl -> predict\_demo.py

Questa distinzione è importante: la demo non modifica la logica reale di controllo, ma rappresenta un primo passo verso una futura estensione data-driven del sistema.

## 3. Posizionamento rispetto all'architettura attuale

Nel progetto OffGas attuale la logica operativa resta centralizzata in **Node-RED**, mentre Arduino e Bridge mantengono un ruolo edge e di comunicazione. La demo AI si inserisce **fuori dal flusso runtime**, come ambiente sperimentale offline.

In pratica:

- il **Bridge** continua a raccogliere e pubblicare telemetria;
- **Node-RED** continua a gestire soglia dinamica, prediction rule-based e stato del sistema;
- la cartella **offgas\_ai\_demo** usa un CSV storico già raccolto per addestrare e testare un modello separato.

Questo approccio permette di sperimentare una prima soluzione ML senza introdurre dipendenze nuove nel sistema reale e senza rischiare di alterare il comportamento dell'architettura già funzionante.

## 4. File presenti nella cartella

La cartella contiene i seguenti elementi principali:

| File                             | Funzione                                          |
|----------------------------------|---------------------------------------------------|
| README_OFFGAS_AI_SIMPLE_DEMO.txt | spiegazione generale della demo                   |
| TODO_LIST_DEMO.txt               | guida operativa per eseguire e presentare la demo |
| WhatToSay.txt                    | supporto testuale per l'esposizione orale         |
| gas_data.csv                     | dataset storico usato come sorgente dei dati      |
| train_randomforest.py            | script di training del modello                    |
| gas_rf_model.pkl                 | modello già addestrato e salvato su disco         |
| predict_demo.py                  | script di inferenza che usa il modello salvato    |

Questa organizzazione è semplice e lineare: separa chiaramente la fase di training da quella di inferenza e rende la demo facile da eseguire anche in sede d'esame.

## 5. Il dataset `gas_data.csv`

Il file `gas_data.csv` rappresenta la base dati della demo.

Secondo la documentazione della cartella, questo file deriva dalla telemetria storica raccolta dal **Bridge** e contiene, nel caso completo, colonne del tipo:

- `timestamp`
- `garage_id`
- `gas`
- `fan_state`

Lo script di training ordina i campioni nel tempo, filtra eventuali righe non valide e, se la colonna `garage_id` è presente, conserva solo i campioni appartenenti a **G1**, cioè il prototipo reale.

La demo usa quindi solo la sequenza temporale del garage reale monitorato, senza coinvolgere direttamente i garage simulati G2-G6.

Questa scelta è coerente con lo scopo della proof of concept: mostrare che i valori storici di G1 possono essere trasformati in un dataset supervisionato utile per una predizione futura.

## 6. Obiettivo del modello

L'obiettivo della demo non è classificare direttamente lo stato del sistema come **SAFE**, **WARNING** o **CRITICAL**.

Il modello viene addestrato per svolgere un compito più elementare e ben definito:

**predire il valore numerico del gas di G1 circa 30 secondi nel futuro.**

Questa è una scelta importante dal punto di vista progettuale:

- l'output del modello è un **numero reale**;
- il confronto con una soglia viene fatto **dopo** la predizione;
- la demo separa quindi la fase di **forecasting** dalla fase di **decisione operativa**.

In questo modo la proof of concept resta semplice, interpretabile e compatibile con una futura integrazione in cui Node-RED potrebbe continuare a mantenere il ruolo di orchestratore finale.

## 7. Scelta del modello: RandomForestRegressor

La demo utilizza un modello di regressione basato su **RandomForestRegressor** della libreria scikit-learn.

Questo modello è stato scelto perché:

- è adatto a predire un valore numerico continuo;
- funziona bene come prima proof of concept su dataset piccoli o moderatamente strutturati;
- richiede poca preparazione infrastrutturale;
- permette di mostrare in modo rapido una pipeline completa di training + inferenza.

Nel file `train_randomforest.py` il modello viene creato con:

- `n_estimators = 100`
- `random_state = 42`

Il fatto che il modello sia una Random Forest significa che la predizione finale deriva dall'aggregazione di più alberi decisionali, con l'obiettivo di migliorare la robustezza rispetto a una singola regola fissa.

## 8. Trasformazione della serie temporale in dataset supervisionato

Uno dei punti più importanti della demo è la trasformazione della serie storica del gas in un problema di apprendimento supervisionato.

La logica è la seguente:

- ogni riga del dataset rappresenta una situazione osservata nel presente;
- a quella situazione viene associato un valore futuro da predire.

Nel file `train_randomforest.py` vengono create cinque feature temporali:

- `gas_now`
- `gas_1_step_ago`
- `gas_2_steps_ago`
- `gas_3_steps_ago`
- `mean_last_4_values`

Queste feature hanno lo scopo di dare al modello un minimo di contesto temporale recente, invece di affidarsi al solo valore istantaneo corrente.

In altre parole, la demo non cerca di imparare la relazione:

`gas attuale -> gas futuro`

ma piuttosto la relazione:

`andamento recente del gas -> gas futuro`

Questo è molto più coerente con un problema di forecasting temporale.

## 9. Orizzonte temporale della predizione

La demo fissa l'orizzonte di predizione a circa **30 secondi**.

Nel file `train_randomforest.py` questo viene rappresentato tramite:

```
PREDICTION_STEPS = 6
```

La documentazione spiega che Arduino invia un campione circa ogni **5 secondi**. Di conseguenza:

```
6 campioni * 5 secondi = circa 30 secondi
```

Il target supervisionato viene costruito spostando in avanti la colonna `gas` di 6 righe:

```
target_gas_in_30s = gas.shift(-6)
```

Questo significa che, per ogni situazione osservata nel presente, il modello viene addestrato a prevedere il valore che comparirà circa sei campioni dopo nel CSV.

## 10. Processo di training in `train_randomforest.py`

Lo script `train_randomforest.py` implementa la pipeline completa di addestramento.

Le operazioni principali sono:

1. caricamento del CSV;
2. conversione dei timestamp;
3. ordinamento temporale dei campioni;
4. filtraggio di G1;
5. creazione delle feature temporali;
6. creazione del target futuro;
7. rimozione delle righe incomplete;
8. suddivisione train/test;
9. training del modello;
10. valutazione con **Mean Absolute Error**;
11. salvataggio del modello su disco.

Il training/test split viene eseguito senza shuffle, così da rispettare l'ordine temporale dei dati. Questo è coerente con la natura sequenziale del problema.

Anche se nel codice compare il calcolo di `split_index`, la separazione effettiva avviene tramite `train_test_split(..., shuffle=False)`, usando quindi il primo 80% come train e l'ultimo 20% come test.

## 11. Valutazione del modello

La metrica usata nella demo è il **Mean Absolute Error (MAE)**.

Il MAE misura di quante unità gas, in media, la predizione si discosta dal valore reale sul test set.

La documentazione della cartella indica un risultato tipico intorno a:

MAE circa 3.76 unità gas

Questo valore viene presentato correttamente come un risultato positivo per una proof of concept, ma non come una validazione definitiva del sistema in scenari critici.

La stessa documentazione sottolinea infatti che il dataset attuale è relativamente stabile, quindi le prestazioni del modello vanno interpretate con cautela.

## 12. Salvataggio del modello: `gas_rf_model.pkl`

Al termine del training, il modello viene salvato nel file binario `gas_rf_model.pkl` tramite `joblib.dump()`.

Questo file non contiene soltanto il modello addestrato, ma un piccolo pacchetto Python che comprende:

- `model`
- `feature_columns`
- `prediction_steps`
- `sample_period_seconds`
- `prediction_horizon_seconds`

Questa scelta è utile perché permette allo script di inferenza di:

- riutilizzare il modello senza riaddestrarlo;
- ricostruire le feature nello stesso ordine del training;
- conoscere e stampare correttamente l'orizzonte temporale della predizione.

## 13. Processo di inferenza in `predict_demo.py`

Lo script `predict_demo.py` rappresenta la parte dimostrativa visibile durante l'esecuzione.

Il suo flusso è il seguente:

1. carica `gas_rf_model.pkl`;
2. legge `gas_data.csv`;
3. filtra e ordina i dati;
4. estrae gli ultimi 4 valori di gas di G1;
5. costruisce le feature richieste dal modello;
6. esegue la predizione;
7. confronta il valore previsto con una soglia fissa di demo;
8. stampa l'output finale.

Per poter funzionare, lo script richiede almeno **4 righe valide** di dati gas, perché la costruzione dell'input usa:

- valore attuale;
- valore di 1 step fa;
- valore di 2 step fa;
- valore di 3 step fa;
- media degli ultimi 4 valori.

L'output finale è pensato per essere leggibile e spiegabile al professore: mostra sia i valori di input sia il valore predetto nel futuro.

## 14. Soglia dimostrativa e `predicted_crossing`

La demo include una soglia fissa impostata nel codice:

```
THRESHOLD = 220.0
```

Questo valore non ha il ruolo della soglia reale del progetto principale. Serve solo a completare la dimostrazione con una logica di confronto finale:

```
predicted_gas >= THRESHOLD
```

Se la condizione è vera, la demo segnala un possibile **predicted crossing** futuro.

È fondamentale chiarire che:

- questa soglia è **manuale e dimostrativa**;
- nel sistema reale la soglia verrebbe calcolata da **Node-RED** in modo dinamico;
- la demo non invia comandi alla ventola;
- la demo non modifica la dashboard.

Questa distinzione è essenziale per non confondere la proof of concept con il comportamento dell'architettura reale.

## 15. Come eseguire la demo

La guida operativa della cartella propone un flusso molto semplice.

### 15.1 Installazione librerie

Dentro la cartella della demo:

```
pip install pandas scikit-learn joblib
```

## 15.2 Training del modello

```
python train_randomforest.py
```

Questo comando crea o aggiorna `gas_rf_model.pkl`.

## 15.3 Esecuzione della predizione

```
python predict_demo.py
```

L'output atteso mostra:

- gli ultimi 4 valori usati;
- il valore attuale;
- la predizione a circa 30 secondi;
- la soglia fissa di demo;
- il risultato booleano `predicted_crossing`.

## 16. Uso di un nuovo `gas_data.csv`

La documentazione della cartella spiega che è possibile sostituire `gas_data.csv` con un file più recente.

Questo porta a due casi distinti.

### 16.1 Nuovo CSV usato solo per inferenza

Se si esegue solo:

```
python predict_demo.py
```

allora:

- il file `.pkl` non viene aggiornato;
- il nuovo CSV viene usato solo per costruire l'input della predizione;
- il modello continua a essere quello addestrato in precedenza.

### 16.2 Nuovo CSV usato anche per riaddestramento

Se si esegue nuovamente:

```
python train_randomforest.py
```

```
python predict_demo.py
```

allora:

- il nuovo CSV entra anche nella fase di training;
- `gas_rf_model.pkl` viene rigenerato;
- le predizioni successive useranno il modello aggiornato.

Questa distinzione è molto importante dal punto di vista concettuale, perché separa chiaramente **inference** e **retraining**.

## 17. Cosa è reale e cosa è dimostrativo nella demo

La cartella contiene sia elementi reali sia elementi volutamente semplificati.

### 17.1 Elementi reali

Sono reali:

- l'uso dei dati storici raccolti dal progetto;

- la pipeline di training;
- il modello Random Forest effettivamente addestrato;
- il salvataggio del modello in `.pkl`;
- la predizione numerica ottenuta sugli ultimi dati disponibili.

## 17.2 Elementi dimostrativi

Sono invece semplificati o fittizi:

- la soglia fissa `THRESHOLD`;
- l'esecuzione completamente offline;
- l'assenza di collegamento diretto a Bridge, MQTT o Node-RED;
- l'assenza di attuazione reale della ventola;
- l'assenza di aggiornamenti alla dashboard.

Questa combinazione è coerente con una proof of concept da esame: mostra una pipeline funzionante senza introdurre complessità di integrazione premature.

## 18. Limiti intenzionali della demo

La documentazione della cartella dichiara chiaramente i limiti principali della proof of concept:

- usa solo i dati di **G1**;
- lavora su un dataset relativamente stabile;
- usa una soglia fissa nel codice;
- non è collegata ai flussi live del sistema;
- non prende decisioni operative reali;
- non controlla ventole o dashboard.

Questi limiti non rappresentano un difetto del progetto, ma una scelta metodologica precisa: dimostrare la fattibilità della pipeline di forecasting con il minimo numero possibile di dipendenze.

## 19. Possibile evoluzione futura

Dal punto di vista architetturale, questa demo suggerisce una possibile evoluzione futura del progetto OffGas.

Una futura integrazione lato server potrebbe seguire questa logica:

1. il Bridge continua a raccogliere telemetria storica;
2. i dati vengono salvati in un archivio aggiornato;
3. un servizio Python AI allena o aggiorna periodicamente il modello;
4. Node-RED invia i valori recenti al servizio AI;
5. il servizio restituisce una predizione futura;
6. Node-RED usa la predizione insieme alla soglia dinamica e alla logica di ventilazione.

In questa prospettiva, la cartella `offgas_ai_demo` non va letta come componente già integrato, ma come **primo prototipo concettuale** di una futura estensione ML del backend.

## 20. Sintesi finale

La cartella `offgas_ai_demo` è una demo separata che mostra come i dati storici prodotti da OffGas possano essere trasformati in una prima pipeline di **Machine Learning predittivo**.

Dal punto di vista funzionale, la demo:

- legge un dataset storico di `G1`;
- costruisce feature temporali semplici;
- addestra un `RandomForestRegressor`;



- salva il modello in `gas_rf_model.pkl`;
- usa il modello per predire il valore del gas a circa 30 secondi;
- confronta la predizione con una soglia dimostrativa fissa.

Dal punto di vista architetturale, la demo:

- non modifica il progetto principale;
- non sostituisce la logica di Node-RED;
- non agisce sul runtime reale;
- dimostra soltanto la fattibilità di una futura estensione AI basata sui dati storici raccolti dal sistema.

In questo senso, `offgas_ai_demo` rappresenta un passaggio molto utile dal punto di vista didattico e progettuale: non è ancora una componente di produzione, ma è una dimostrazione concreta che OffGas può evolvere da logica rule-based a supporto predittivo data-driven.